

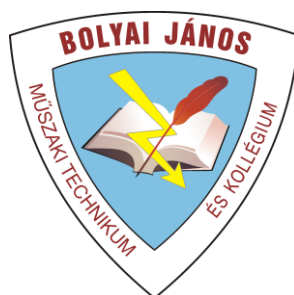


Backstage Gear

Szerzők:

Nagy Bálint Zsombor és Rózsavölgyi Zétény István

Konzulens tanárok: Dorogházi Donát és Gombos Szabolcs



BMSzC Bolyai János Műszaki Technikum és Kollégium

Tartalomjegyzék

1. Bevezetés	4
2. Specifikáció	4
2.1 Rövid leírás	4
2.2 Jogosultságok, szerepkörök	5
2.2.1 Nem regisztrált felhasználó:	5
2.2.2 Felhasználó:	5
2.2.3 Adminisztrátor:	5
2.3 Követelmények	5
2.3.1 Funkcionális	5
2.3.1.1 Regisztráció, bejelentkezés, jelszókezelés:	5
2.3.1.2 Hirdetésfeltöltés, módosítás (adatok: név, leírás, kategória, ár, állapot, képek):	6
2.3.1.3 Termékek szűrése:	6
2.3.1.4 Termékek kosárba helyezése és vásárlási folyamat:	6
2.3.1.5 Üzenetküldés a hirdetőkhöz:	6
2.3.1.6 Profilok szerkesztése:	7
2.3.1.7 Profilok értékelése:	7
2.3.1.8 Hirdetés jelentése és moderálása:	7
2.3.2 Nem funkcionális	7
2.4 Célok, célkitűzések	7
2.5 Felhasználási/alkalmazási területek	8
2.5.1 Személyek közötti adásvétel támogatása:	8
2.5.2 Használt termékek böngészése és vásárlása:	8
2.5.3 Eladók számára értékesítési platform biztosítása:	8
2.5.4 Adminisztrátori funkciók:	8
2.5.5 Tipikus alkalmazási környezetek:	8
2.6 Fejlesztési Módszertan	8
2.7 Csoport tagjainak Szerepköre	9
3. Fejlesztői dokumentáció	9
3.1 Fejlesztői környezet	9
3.2 Fejlesztői eszközök	9
3.3 Adatbázis	10
3.3.1 Táblák	10
3.3.2 Tesztfelhasználók	10
3.3.3 Adatbázis szerkezete, mezői	11
3.3.3.1 Users tábla	11

3.3.3.2 Profiles tábla.....	12
3.3.3.3 Profile_votes tábla	12
3.3.3.4 Messages tábla.....	13
3.3.3.5 Brands tábla.....	13
3.3.3.6 Categories tábla	13
3.3.3.7 Brand_categories tábla.....	13
3.3.3.8 Items tábla	14
3.3.3.9 Used_items tábla.....	14
3.3.3.10 Advertisements tábla.....	14
3.3.3.11 Files tábla	15
3.3.3.12 Ad_files tábla.....	15
3.3.3.13 Carts tábla.....	15
3.3.4 Táblák közötti kapcsolatok.....	16
3.4 Rest API dokumentáció	18
3.5 Frontend komponensek bemutatása.....	25
3.5.1 Login komponens	25
3.5.2 Registration komponens.....	26
3.5.3 Filter komponens	27
4. Tesztelői dokumentáció	28
4.1 Tesztelés célja, menete	28
4.2 Tesztelési környezet	28
4.3 Manuális tesztek.....	28
4.2.1 Bejelentkezés tesztek	30
4.2.2 Regisztráció tesztek	32
4.2.3 Új hirdetés tesztek.....	33
4.4 Automatizált tesztek	34
4.4.1 Az automatizált tesztekhez használt eszközök és technológiák	34
4.4.2 Példák automatizált tesztekre	35
4.4.2.1 E2E példateszt	35
4.4.2.2 Unit példateszt	36
4.4.2.3 Frontend komponensszintű példa	38
4.4.2.4 Frontend oldalszintű példa	38
5. Felhasználói dokumentáció	39
5.1 Böngészés	39
5.2 Regisztráció.....	39
5.3 Bejelentkezés	40

5.4 Hirdetés kívánság listára helyezése	40
5.5 Kívánságlista megtekintése	40
5.6 Új hirdetés létrehozása.....	41
5.7 Hirdetések módosítása, törlése, jelentése.....	41
5.8 Üzenetek kezelése	42
5.9 Profil kezelése	43
5.10 Profil értékelése.....	43
5.11 Kijelentkezés	44
6. Összefoglalás.....	44
6.1 Rövid értékelés:.....	44
7. Kitekintés.....	44
8. Irodalomjegyzék/Bibliográfia	45
9. Mellékletek	45

1. Bevezetés

A dokumentáció szerzői:

- Nagy Bálint Zsombor
- Rózsavölgyi Zétény István

2. Specifikáció

2.1 Rövid leírás

A projektünk egy használt árucikk webáruház / webapplikáció, ahol használt hangszereket, hangtechnikai eszközöket lehet árulni és vásárolni. Egy, az oldalra látogató személynek nem szükséges regisztrálnia vagy bejelentkeznie, amennyiben csak böngészni szeretne a termékek között. Viszont, ha úgy dönt, hogy vásárolni szeretne vagy fel szeretne tenni egy hirdetést, akkor a regisztráció / bejelentkezés szükségszerűvé válik. A felhasználók fel tudnak tenni hirdetéseket, jelezni egy hirdetés iránt az érdeklődésüket és akár kosárba is tenni a kiválasztott termékeket. Továbbá fel tudják venni a kapcsolatot egy hirdetővel, üzenetet küldeni neki, illetve a vásárlás során értékelni az adott profilt. Továbbá lehetőségük van szűrni egy adott árra, termékre, termékcsoportha vagy márkára. A felhasználók üzenetet is küldhetnek a hirdetőknél, akik egy postaládában megtekinthetik azokat. Továbbá lehetőség van a profilok szerkesztésére pl.: felhasználónév, profilkép, jelszó, lakcím. Amennyiben valaki vásárolni szeretne, abban az esetben lehetőség van egy kosárba tenni a megvásárolni kívánt terméket / termékeket. A felhasználók az oldal megnyitásakor egy számukra aktuális, ajánlott hirdetést / hirdetéseket láthatnak. Ha esetleg egy nem kívánt vagy kifogásolható tartalomra bukkan egy felhasználó, akkor az adott hirdetést jelentheti, amit egy adminisztrátor felhasználó fog moderálni. Az adminisztrátor képes törölni egy hirdetést vagy akár egy profilt is.

2.2 Jogosultságok, szerepkörök

2.2.1 Nem regisztrált felhasználó:

- Böngészés a weboldalon.
- Termékek keresése, szűrése ár, kategória, márka alapján.

2.2.2 Felhasználó:

- Hirdetések feltöltése.
- Termékek keresése, szűrése, kosárba helyezése.
- Üzenetküldés az eladónak.
- Profilok értékelése.
- Hirdetések jelentése.

2.2.3 Adminisztrátor:

- Felhasználók és hirdetések kezelése.
- Szabálysértő tartalmak moderálása.
- Profilok és hirdetések törlése.

2.3 Követelmények

2.3.1 Funkcionális

2.3.1.1 Regisztráció, bejelentkezés, jelszókezelés:

A weboldal tartalmaz egy regisztráció / bejelentkezés oldalt, ahol a felhasználó megadhatja a megfelelő mezőbe a felhasználónevét, email címét a jelszóval, telefonszámát, születési idejét és a lakcímét. Mindegyik mezőnek a kitöltése szükséges a sikeres regisztrációhoz vagy bejelentkezéshez. Fontos, hogy egy felhasználónak nem szükséges bejelentkeznie, amennyiben csak böngészési céllal veszi igénybe az oldalt.

2.3.1.2 Hirdetésfeltöltés, módosítás (adatok: név, leírás, kategória, ár, állapot, képek):

Amennyiben egy felhasználó be van jelentkezve a profiljába, elérhető a számára egy „Új hirdetés” fül, amely lehetőséget nyújt a számára, hogy hirdetést tegyen fel a weboldalra. Egy új hirdetés létrehozásánál szükséges megadni valamilyen rövid leírást az eladandó termékről, annak kategóriáját (pengetős, ütős, billentyűs, vonós, fúvós), márkáját, valamilyen elérhetőséget, melynek segítségével egy vásárló kapcsolatba tud lépni egy hirdetővel, legalább egy képet a termékről, a termék árát és annak állapotát (új, használt, sérült). Ha változnak egy hirdetés információi, akkor abban az esetben lehetőség van egy hirdetést megváltoztatni.

2.3.1.3 Termékek szűrése:

Lehetőség van a termékek szűrésére ár, márka, kategória és állapot alapján.

2.3.1.4 Termékek kosárba helyezése és vásárlási folyamat:

Abban az esetben, ha egy felhasználó be van jelentkezve, lehetősége van termékeket vásárolni. A kívánt terméket / termékeket behelyezheti egy kosárba. A kosárba helyezést követően elindítható a vásárlási folyamat, melynek során a vásárlónak meg kell adnia az adatait a megfelelő mezőbe (név, email cím, telefonszám, lakcím, postázási cím). Fontos, hogy mindegyik adat megadása kötelező, ellenben nem kivitelezhető a vásárlás. Mind sikeres, mind sikertelen vásárlás esetén az oldal küld egy visszaigazoló üzenetet. Sikeres vásárlás esetén a hirdetés automatikusan inaktívvá válik, ám annak az oldalról való eltávolítása a hirdető felhasználó felelőssége.

2.3.1.5 Üzenetküldés a hirdetőknak:

Ha egy potenciális vásárlónak kérdése lenne egy termékkel vagy hirdetéssel kapcsolatban, küldhet üzenetet a hirdetőnek. Ez a hirdető profiljának megtekintése esetében kivitelezhető. Az üzenetet a hirdető a személyes postaládájában megtekintheti és válaszolhat rá. Erre a személyes profil beállításoknál van lehetőség.

2.3.1.6 Profilok szerkesztése:

A profilok szerkesztésére is van lehetőség, ha egy felhasználó meg szeretné változtatni az adatait (felhasználónév, jelszó, telefonszám, profilkép, lakcím). Az email cím állandó, az nem változtatható meg. Ez a funkció a profilunk megtekintése esetében érhető el.

2.3.1.7 Profilok értékelése:

Egy bejelentkezett felhasználónak lehetősége van értékelni egy hirdető profilt (up vote, down vote). Ez a profil megtekintésének esetében kivitelezhető. Ezek az értékelések láthatók a többi felhasználó számára és így eldönthetik mennyire bíznak meg egy adott hirdetőben.

2.3.1.8 Hirdetés jelentése és moderálása:

Ha egy felhasználó egy olyan hirdetésre bukkan, amely szerinte kifogásolható sértő vagy nem releváns az oldal céljainak szempontjából, akkor jelentheti az adott hirdetést. Erre a hirdetés megtekintésének esetében van lehetőség egy külön erre a célra szolgáló gombbal. Ezeket a jelentéseket az adminisztrátor felhasználó értékeli ki és ennek a segítségével dönthet egy adott hirdetés vagy hirdető törléséről. Fontos, hogy egy felhasználó csak abban az esetben tehet ilyen jelentést, ha be van jelentkezve a fiókjába.

2.3.2 Nem funkcionális

- Reszponzív design (mobil és asztali támogatás).
- Gyors oldalbetöltés és keresés.
- Biztonságos adatkezelés és titkosítás. (Titkosított adat pl.: jelszavak)

2.4 Célok, célkitűzések

A cél a tervezett funkciók sikeres megvalósítása egy stabil és jól működő rendszer kialakításával. Kiemelt figyelmet kap a fejlesztés során felmerülő hibák kezelése, valamint azok megelőzése, hogy a végeredmény megbízható és könnyen karbantartható legyen. Hosszú távú célként a projekt

továbbfejlesztése is szerepel, amely során új funkciók kerülhetnek beépítésre a felhasználói igények és a technológiai fejlődés függvényében.

2.5 Felhasználási/alkalmazási területek

2.5.1 Személyek közötti adásvétel támogatása:

- A felhasználók regisztrálhatnak, és saját fiókkal termékeket tölthetnek fel eladásra.
- Más regisztrált felhasználók ezeket megvásárolhatják, értékelhetik az eladókat.

2.5.2 Használt termékek böngészése és vásárlása:

- A látogatók árak, márkák, kategóriák szerint böngészhetik a termékeket.
- Lehetőség van hirdetésre és vásárlásra regisztrációval.

2.5.3 Eladók számára értékesítési platform biztosítása:

- A rendszer lehetővé teszi a termékek adatainak (ár, állapot, leírás, képek, márkák) feltöltését.

2.5.4 Adminisztrátori funkciók:

- Adminisztrátorok kezelhetik a felhasználókat, hirdetéseket, panaszokat.
- Moderációs eszközök a szabályzat megsértésének kezelésére.

2.5.5 Tipikus alkalmazási környezetek:

- Webes böngésző (desktop és mobil eszközökön).
- Jellemző felhasználók: magánszemélyek, gyűjtők, újrahasznosításban érdekeltek.

2.6 Fejlesztési Módszertan

Agilis – Kan:

- Vizuális tábla használata a feladatok átláthatóságához és nyomon követéséhez.
- Korlátozott mennyiségű feladat kezelése a hatékonyság érdekében.
- Folyamatos fejlesztést és rendszeres visszajelzés.
- A feladatok fontossági sorrend alapján való prioritizálása.

2.7 Csapat tagjainak Szerepköre

- Frontend fejlesztése és tesztelése: **Nagy Bálint Zsombor** és Rózsavölgyi Zétény István
- Backend fejlesztése és tesztelése: **Rózsavölgyi Zétény István** és Nagy Bálint Zsombor
- Specifikáció: Nagy Bálint Zsombor és Rózsavölgyi Zétény István

3. Fejlesztői dokumentáció

3.1 Fejlesztői környezet

A vizsgaremek három fő részből áll: **adatbázis**, **backend** és **frontend**.

- **Adatbázis:** az adatbázist a MySQL segítségével hoztuk létre és kezeltük, aminek a hosztolását XAMPP és Docker használatával végeztük.
- **Backend:** a backend logikát, annak frontend-el és adatbázissal való kommunikációját Node.js futtatókörnyezet segítségével, annak Express.js keretrendszerének használatával valósítottuk meg, TypeScript nyelven.
- **Frontend:** az oldal megjelenését, és a backend-el való kommunikációt a JavaScript React könyvtárának segítségével oldottuk meg. Az oldal design részét és annak reszponzív mivoltát Bootstrap használatával valósítottuk meg.

3.2 Fejlesztői eszközök

- **Fejlesztői környezet:** Visual Studio Code
- **Verziókezelő rendszer:** GitHub
- **Adatbázis kezelő rendszerek:** Docker

3.3 Adatbázis

3.3.1 Táblák

Users: Ez a tábla tárolja az oldalra regisztrált felhasználók adatait.

Profiles: Ez a tábla tárolja a felhasználókhöz tartozó profilok adatait.

Profile_votes: Ez a tábla tárolja a felhasználókhöz tartozó pozitív és negatív értékeléseket.

Messages: Ez a tábla tárolja a felhasználók elküldött és bejövő üzeneteit.

Brands: Ez a tábla tárolja az összes márkát, ami az oldalon megjelenő termékekhez tartozik.

Categories: Ez a tábla tárolja az összes, hirdetésekhez tartozó kategóriát.

Brand_categories: Ez a tábla párosítja össze a márkákat és a kategóriákat.

Items: Ez a tábla tárolja az oldalon hirdetett termékek adatait.

Used_items: Ez a tábla tárolja az oldalon hirdetett használt termékek információit.

Advertisements: Ez a tábla tárolja az összes oldalon megjelenő hirdetés adatait.

Files: Ez a tábla tárolja az oldalra feltöltött, különböző hirdetésekhez tartozó képek adatait.

Ad_files: Ez a tábla kapcsolja össze a hirdetéseket a hozzájuk tartozó képekkel.

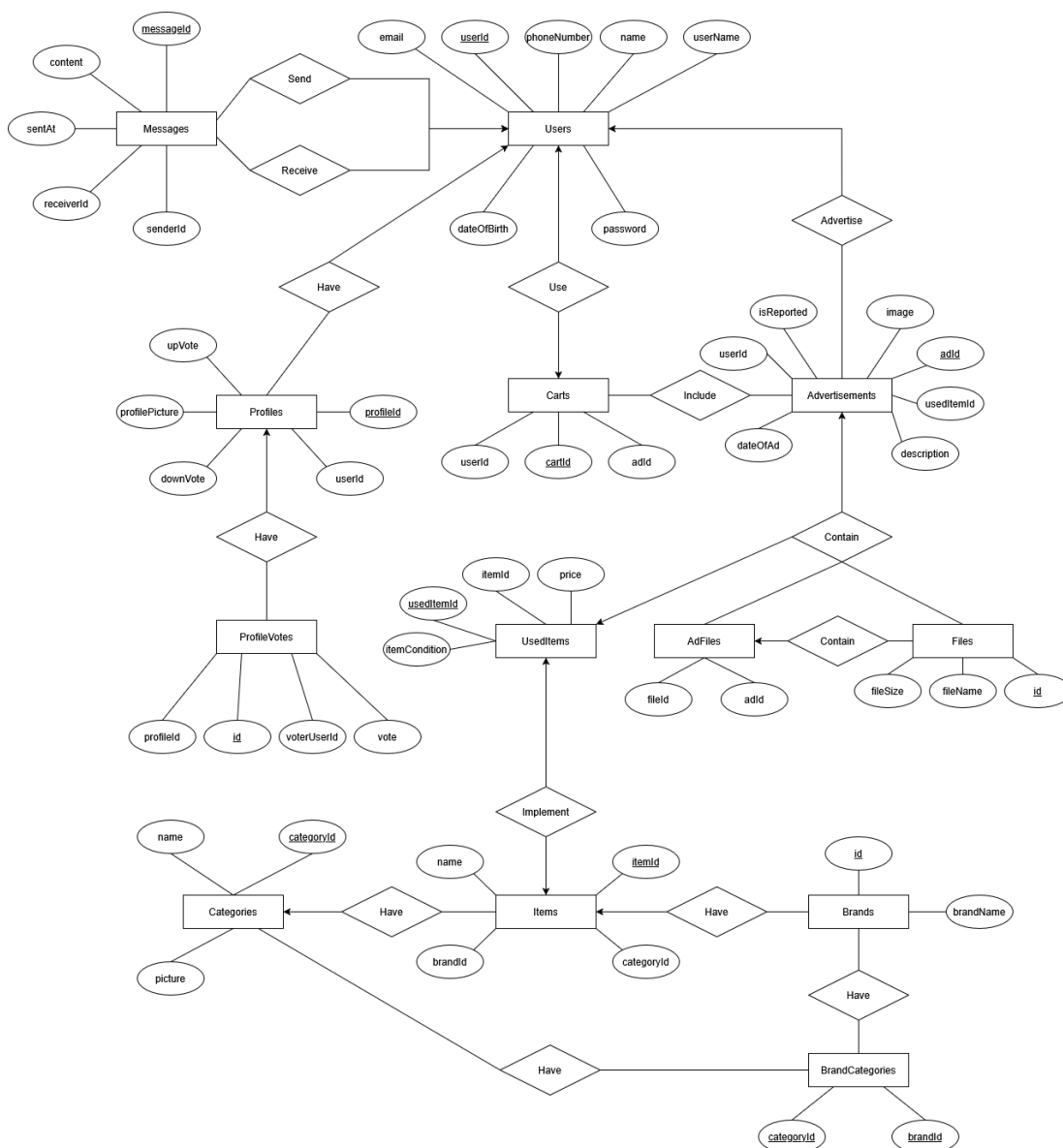
Carts: Ez a tábla tárolja a felhasználók kosárba mentett hirdetéseiknek az adatait.

3.3.2 Tesztfelhasználók

Id	Szerepkör	Felh. név	Email	Jelszó
1	admin	admin	admin@example.com	admin
2	felhasználó	nagy.eva	eva.nagy@example.com	jelszo1

3	felhasználó	toth.sandor	sandor.toth@example.com	jelszo2
4	felhasználó	major.zsuzsa	zsuzsa.major@example.com	jelszo3
5	felhasználó	janos.peter	peter.janos@example.com	jelszo4

3.3.3 Adatbázis szerkezete, mezői



1. Ábra: Az adatbázis ER modellje

3.3.3.1 Users tábla

Id: INTEGER, AUTO_INCREMENT, elsődleges kulcs, a felhasználó egyedi azonosítóját tartalmazza

Is_admin: TINYINT, DEFAULT 0, igaz/hamis érték, a felhasználó szerepkörét tartalmazza (1 – admin, 0 - felhasználó)

Name: VARCHAR, kötelező, a felhasználó neve

Username: VARCHAR, kötelező, egyedi, a felhasználónevet tartalmazza

Email: VARCHAR, kötelező, egyedi, a felhasználó email címe

Phone_number: VARCHAR, kötelező, a felhasználó telefonszáma

Date_of_birth: DATE, a felhasználó születési ideje

Password: VARCHAR, kötelező, SHA2 technológiával titkosított és sózott, a felhasználó jelszava

3.3.3.2 Profiles tábla

Id: INTEGER, AUTO_INCREMENT, elsődleges kulcs, a profil egyedi azonosítóját tartalmazza

User_id: INTEGER, kötelező, idegen kulcs, a profilhoz tartozó felhasználó egyedi azonosítója

Profile_picture: VARCHAR, kötelező, DEFAULT “default-profile-picture.jpg”, a felhasználó profilképe

3.3.3.3 Profile_votes tábla

Id: INTEGER, kötelező, idegen kulcs, a szavazathoz tartozó felhasználó egyedi azonosítója

Profile_id: INTEGER, kötelező, idegen kulcs, a szavazatot kapott profil egyedi azonosítója

Voter_user_id: INTEGER, kötelező, idegen kulcs, a szavazatot leadó profil egyedi azonosítója

Vote: TINYINT, kötelező, a szavazat típusa (1 – up vote, 0 – down vote)

3.3.3.4 Messages tábla

Id: INTEGER, AUTO_INCREMENT, elsődleges kulcs, az üzenet egyedi azonosítóját tartalmazza

Sender_id: INTEGER, kötelező, idegen kulcs, az üzenetet küldő felhasználó egyedi azonosítója

Receiver_id: INTEGER, kötelező, idegen kulcs, az üzenetet fogadó felhasználó egyedi azonosítója

Content: TEXT, kötelező, az üzenet tartalma

Sent_at: DATETIME, kötelező, DEFAULT NOW(), az üzenet elküldésének az ideje

3.3.3.5 Brands tábla

Id: INTEGER, AUTO_INCREMENT, elsődleges kulcs, a márka egyedi azonosítóját tartalmazza

Brand_name: VARCHAR, kötelező, egyedi, a márka neve

3.3.3.6 Categories tábla

Id: INTEGER, AUTO_INCREMENT, elsődleges kulcs, a kategória egyedi azonosítóját tartalmazza

Name: VARCHAR, kötelező, a kategória megnevezése

Picture: VARCHAR, kötelező, a kategóriához tartozó borítóképet tartalmazza

3.3.3.7 Brand_categories tábla

Brand_id: INTEGER, kötelező, idegen kulcs, a márka egyedi azonosítója, az összetett kulcs egyik része

Category_id: INTEGER, kötelező, idegen kulcs, a kategória egyedi azonosítója, összetett kulcs másik része

3.3.3.8 Items tábla

Id: INTEGER, AUTO_INCREMENT, elsődleges kulcs, a termék egyedi azonosítóját tartalmazza

Category_id: INTEGER, kötelező, idegen kulcs, a kategória egyedi azonosítója

Brand_id: INTEGER, kötelező, idegen kulcs, a márka egyedi azonosítója

Name: VARCHAR, a termék megnevezése

3.3.3.9 Used_items tábla

Id: INTEGER, AUTO_INCREMENT, elsődleges kulcs, a használt termék egyedi azonosítóját tartalmazza

Item_id: INTEGER, kötelező, idegen kulcs, a termék egyedi azonosítója

Price: INTEGER, a használt termék ára

Item_condition: VARCHAR, kötelező, a használt termék állapota

3.3.3.10 Advertisements tábla

Id: INTEGER, AUTO_INCREMENT, elsődleges kulcs, a hirdetés egyedi azonosítóját tartalmazza

User_id: INTEGER, kötelező, idegen kulcs, a hirdetéshez tartozó felhasználó egyedi azonosítója

Used_item_id: INTEGER, kötelező, idegen kulcs, a hirdetéshez tartozó használt termék egyedi azonosítója

Is_reported: TINYINT, DEFAULT false, igaz/hamis érték, egy hirdetés jelentésének logikai értékét tartalmazza, (0 – nem jelentett, 1- jelentett)

Description: TEXT, kötelező, a hirdetés részletes leírása

Date_of_ad: DATETIME, kötelező, DEFAULT NOW(), a hirdetés létrehozásának időpontja

3.3.3.11 Files tábla

Id: INTEGER, AUTO_INCREMENT, elsődleges kulcs, a fájl egyedi azonosítóját tartalmazza

File_name: VARCHAR, kötelező, a fájl neve

File_size: INTEGER, kötelező, a fájl mérete

3.3.3.12 Ad_files tábla

Ad_id: INTEGER, kötelező, idegen kulcs, a fájlhoz tartozó hirdetés egyedi azonosítója

File_id: VARCHAR, idegen kulcs, a fájl neve

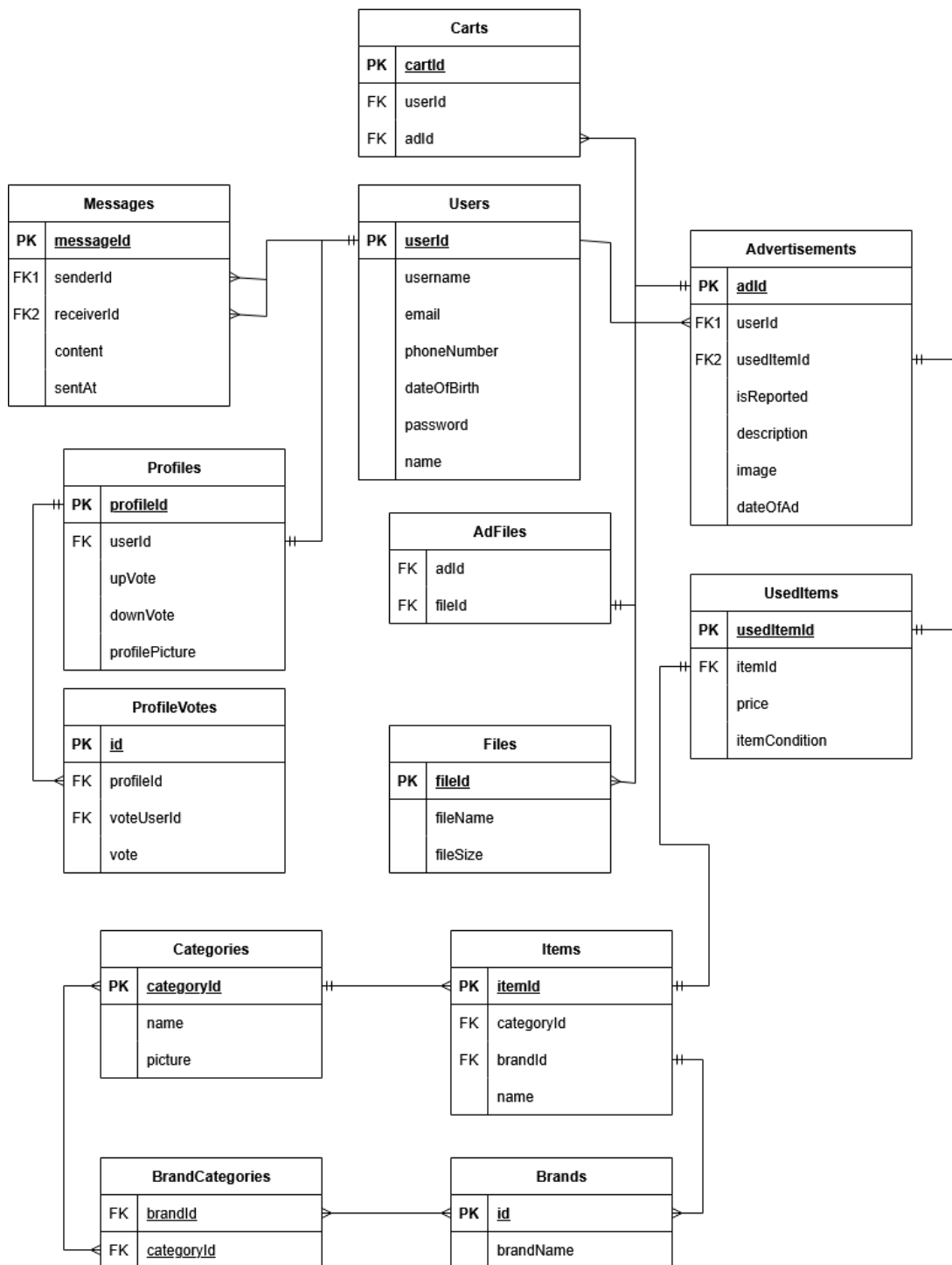
3.3.3.13 Carts tábla

Id: INTEGER, AUTO_INCREMENT, elsődleges kulcs, a tartalom egyedi azonosítóját tartalmazza

User_id: INTEGER, kötelező, idegen kulcs, a kosárhoz tartozó felhasználó egyedi azonosítója

Ad_id: INTEGER, kötelező, idegen kulcs, a kosárba helyezett hirdetéshez tartozó egyedi azonosító

3.3.4 Táblák közötti kapcsolatok

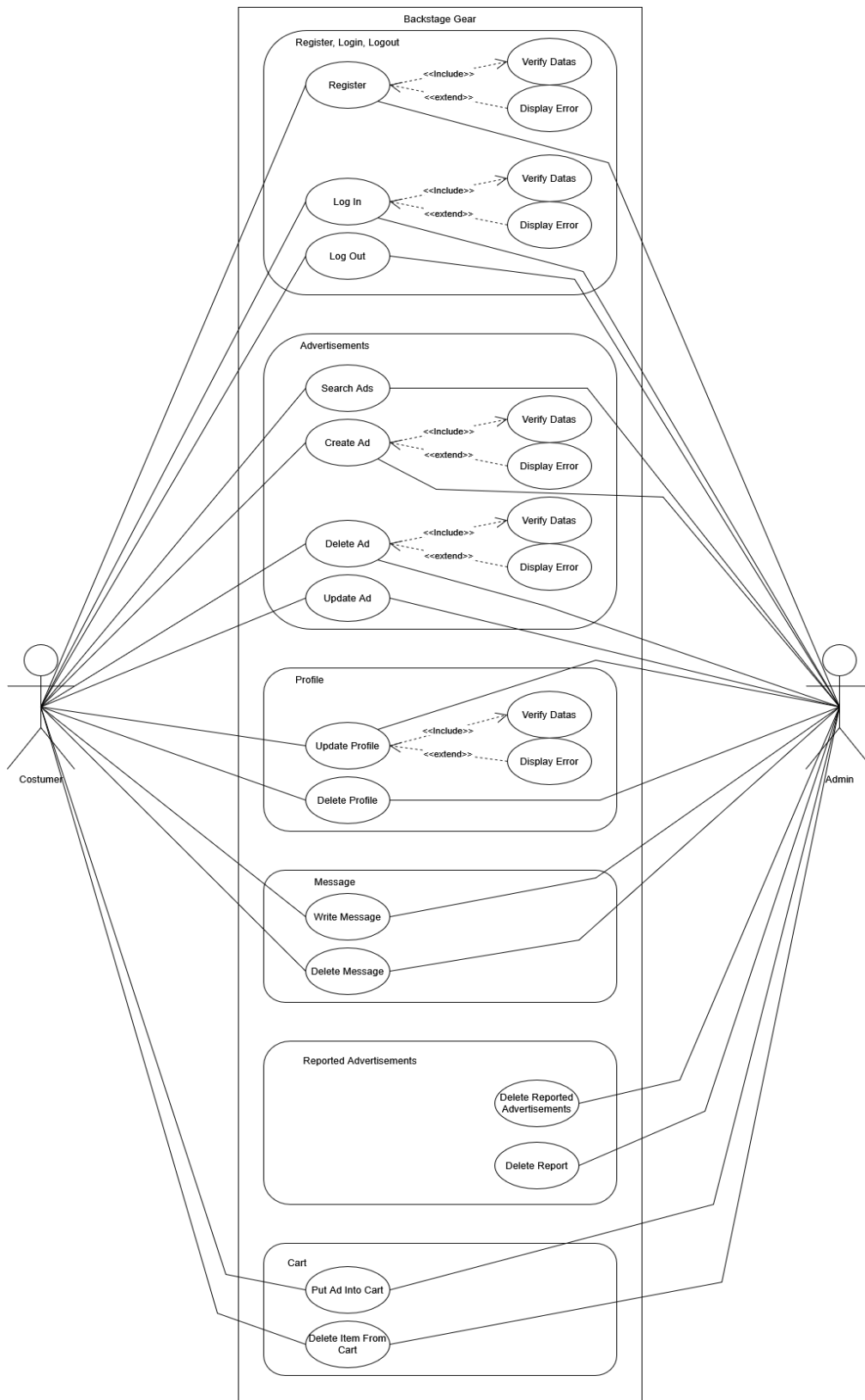


2. Ábra: Az adatbázis ERD modellje

- Users és Messages táblák között 1:N (egy a többhöz): egy felhasználóhoz több üzenet is tartozhat, de egy bizonyos üzenet csak egy felhasználóhoz tartozhat.

- Users és Profiles táblák között 1:1 (egy az egyhez): egy felhasználónak csak egy profilja van és egy profil csak egyetlen felhasználóhoz tartozhat.
- Profiles és Profile_votes táblák között 1:N (egy a többhöz): egy profil több szavazatot is leadhat, de egy adott szavazat csak egyetlen profilhoz tartozhat.
- Users és Carts táblák között 1:1 (egy az egyhez): egy felhasználónak csak egy kosara lehet és egy adott kosár csak egy felhasználóhoz tartozhat.
- Users és Advertisements táblák között 1:N (egy a többhöz): egy felhasználó akár több hirdetést is létrehozhat, de egy adott hirdetés csakis egyetlen felhasználóhoz tartozik.
- Advertisements és Used_items táblák között 1:1 (egy az egyhez): egy hirdetéshez csak egy használt termék tartozhat és egy adott használt termék csak egy hirdetéshez tartozhat.
- Used_items és Items táblák között 1:1 (egy az egyhez): egy használt termékhez csak egy termék tartozhat és egy termék csak egy használt termékhez tartozhat.
- Categories és Items táblák között 1:N (egy a többhöz): egy kategóriához több termék is tartozhat, de egy adott termékhez csak egy kategória tartozhat.
- Brands és Items táblák között 1:N (egy a többhöz): egy márka több termékhez is tartozhat, de egy adott terméknek csakis egy márkája lehet.
- Brands és Brand_categories között N:M (több a többhöz): egy márka több kategóriába is tartozhat és egy kategóriába akár több márka is tartozhat.
- Categories és Brand_categories között N:M (több a többhöz): egy kategória több márkához is tartozhat és egy márka több kategóriához is tartozhat.
- Advertisements és Ad_files között 1:1 (egy az egyhez): egy hirdetéshez csak egy fájl tartozhat és egy adott fájl csak egy hirdetéshez tartozhat.
- Ad_files és Files között 1:1 (egy az egyhez): egy hirdetéshez csak egy fájl tartozhat és egy adott fájl csak egy hirdetéshez tartozhat.

3.4 Rest API dokumentáció



3. Ábra: Sequence Diagram

Metódus: GET

URL: /backstagegear/ads

Leírás: lekéri az összes hirdetést

Válasz (200 OK):

```
[
  {
    "id": 1,
    "name": "Fender American Professional II Stratocaster HSS, RW, Dark Night",
    "description": "Öreg Fender Strató, kifejezetten vékony, kényelmes nyakkal, tokkal és vonóval eladóvá vált. A nyakon van egy profin bedugózott furat, ezért ennyi az ára. Extra Jumbó, alig használt bundokkal, elől és hátul YJM Seymour Duncan, középen Kinman hangszedők. Baráti, 3,5 Kg súly!",
    "price": 860000,
    "files": [
      "http://localhost:3000/ad-pictures/fender-american-professional-ii.jpg"
    ]
  },
]
```

4. Ábra: GET példa válasz az összes hirdetés lekéréséhez

Lehetséges státuszkódok:

- 200 OK
- 404 Not Found – Nincsenek lekérendő hirdetések.
- 500 Internal Server Error – Szerverhiba.

A felhasználók ezen a végponton keresztül kérhetik le az oldalon található összes létrehozott hirdetést. Ezen hirdetések adatainak a lekéréséhez nincsen szükség regisztrációra vagy bejelentkezésre, ezek az információk mindenki számára elérhetőek.

Metódus: POST

URL: /backstagegear/me/new_ad

Leírás: létrehoz egy új hirdetést

Autentikáció: JWT token szükséges:

```
Headers: {  
  
    x-access-token: "JWT-token"  
  
}
```

Request body:

```
{  
  "categoryId": 5,  
  "brandId": 3,  
  "itemName": "Ampeg SVT-410HLF",  
  "price": 383380,  
  "condition": "új",  
  "description": "Dobozával együtt, új állapotban árurom. Budapesten megtekinthető, kipróbálható, megvehető."  
}
```

5. Ábra: POST példa kérés az új hirdetés létrehozásához

Válasz (201 Created):

```
{  
  "message": "Hirdetés sikeresen létrehozva.",  
  "adId": 7  
}
```

6. Ábra: POST példa válasz sikeres hirdetés létrehozásnál

Lehetséges státuszkódok:

- 201 Created - Hirdetés sikeresen létrehozva.
- 400 Bad Request – Nem küldte el az adatokat.
- 400 Bad Request - Hibásan vagy hiányosan megadott adatok.
- 401 Unauthorized – Token szükséges.
- 500 Internal Server Error – Hiba történt a hirdetés létrehozása során.

A felhasználók ezen a végponton keresztül tudnak új hirdetéseket létrehozni. Egy hirdetés létrehozásához szükséges regisztráció vagy bejelentkezés. Egy hirdetés létrehozásánál kötelező megadni az alábbi mezőket: kategória, márka, a termék neve/megnevezése, a termék ára, állapota és a hirdetés részletes leírása.

Metódus: PATCH

URL: /backstagegear/me/my_ads/update_ad/:adId

Leírás: frissíti egy adott hirdetés adatait

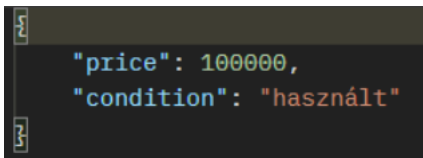
Autentikáció: JWT token szükséges:

```
Headers: {  
    x-access-token: "JWT-token"  
}
```

Request paraméterek:

- adId: 1

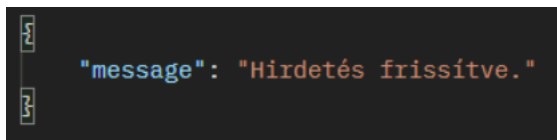
Request Body:



```
{  
  "price": 100000,  
  "condition": "használt"  
}
```

7. Ábra: PATCH példa kérés
hirdetés módosításhoz

Válasz (201 Created):



```
{  
  "message": "Hirdetés frissítve."  
}
```

8. Ábra: PATCH példa válasz hirdetés
sikeres módosításánál

Lehetséges státuszkódok:

- 200 OK - Hirdetés frissítve.
- 400 Bad Request - Nem megfelelő formátumú azonosító.
- 401 Unauthorized – Token szükséges.
- 403 Forbidden - Nincs jogosultságod a hirdetés módosításához.
- 404 Not Found - Nincs ilyen azonosítójú hirdetés.
- 500 Internal Server Error - Hiba történt a hirdetés frissítése során.

A felhasználók ezen a végponton keresztül frissíthetik egy adott hirdetésük adatait pl.: leírás, kép, megnevezés, ár. Ennek végrehajtásához szükséges, hogy a felhasználó legyen bejelentkezve és legyen legalább egy hirdetése.

Metódus: DELETE

URL: /backstagegear/me/my_ads/:adId

Leírás: töröl egy adott hirdetést

Autentikáció: JWT token szükséges:

```
Headers: {  
    x-access-token: "JWT-token"  
}
```

Request paraméterek:

- adId: 7

Válasz (204 No Content)

Lehetséges státuszkódok:

- 204 No Content
- 400 Bad Request - Nem megfelelő formátumú azonosító.
- 401 Unauthorized – Token szükséges.
- 500 Internal Server Error – Szerverhiba.

A felhasználók ezen a végponton keresztül törölhetik hirdetéseiket. Ennek teljesítéséhez szükséges, hogy a felhasználó be legyen jelentkezve és legyen legalább egy hirdetése, amit tud törölni.

Metódus: GET

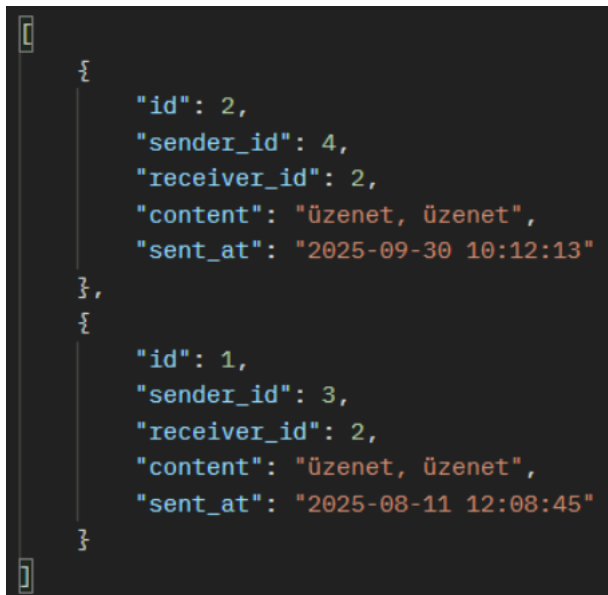
URL: /backstagegear/me/incoming_messages

Leírás: lekéri a felhasználó bejövő üzeneteit

Autentikáció: JWT token szükséges

```
Headers: {  
    x-access-token: "JWT-token"  
}
```

Válasz (200 OK):



```
[  
  {  
    "id": 2,  
    "sender_id": 4,  
    "receiver_id": 2,  
    "content": "Üzenet, üzenet",  
    "sent_at": "2025-09-30 10:12:13"  
  },  
  {  
    "id": 1,  
    "sender_id": 3,  
    "receiver_id": 2,  
    "content": "Üzenet, üzenet",  
    "sent_at": "2025-08-11 12:08:45"  
  }  
]
```

9. Ábra: GET példaválasz bejövő üzenetek lekérésénél

Lehetséges státuszkódok:

- 200 OK
- 401 Unauthorized – Token szükséges.
- 404 Not Found - Ez a felhasználó nem létezik.
- 404 Not Found - Nincsenek beérkező üzenetek.
- 500 Internal Server Error – Szerverhiba.

A felhasználók ezen a végponton keresztül képesek lekérni a bejövő üzeneteiket.

Metódus: POST

URL: /backstagegear/me/new_message/:recId

Leírás: elküld egy új üzenetet

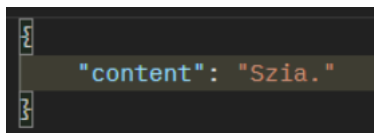
Autentikáció: JWT token szükséges

```
Headers: {  
    x-access-token: "JWT-token"  
}
```

Request paraméterek:

- recId: 3

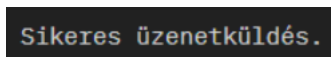
Request Body:



```
{  
  "content": "Szia."  
}
```

*10. Ábra: POST példa kérés
üzenetküldésnél*

Válasz (201 Created):



```
Sikeres üzenetküldés.
```

*11. Ábra: POST példa
válasz sikeres
üzenetküldésnél*

Lehetséges státuszkódok:

- 201 Created - Sikeres üzenetküldés.
- 400 Bad Request - Nem megfelelő formátumú azonosító.
- 400 Bad Request - Nem küldte el az adatokat.
- 400 Bad Request - Hibásan vagy nem megfelelően megadott adatok.
- 401 Unauthorized – Token szükséges.
- 404 Not Found - Ez a felhasználó nem létezik.
- 404 Not Found - Nem sikerült elküldeni az üzenetet.
- 500 Internal Server Error – Szerverhiba.

A felhasználók ezen a végponton keresztül írhatnak új üzenetet egy általuk kiválasztott másik felhasználónak.

3.5 Frontend komponensek bemutatása

A frontend egy React alapú Single Page Application (SPA), amely az alkalmazás felhasználói felületének megjelenítéséért és a felhasználó i interakciók kezeléséért felelős. Az SPA megközelítés lényege, hogy az alkalmazás egyetlen HTML oldalon működik, és a tartalom dinamikusan frissül a felhasználói műveletek hatására, teljes oldalújrátöltés nélkül. A frontend feladatai közé tartozik az eseménykezelés, valamint az adatok struktúrált megjelítése. A rendszer a backenddel REST API-n keresztül kommunikál, amely során az adatok küldése és fogadása JSON formátumban történik. Ez lehetővé teszi az aszinkron adatkérést és -küldést, ezzel biztosítva a felhasználói felület folyamatos frissítését.

3.5.1 Login komponens

A Login komponens az alkalmazás felhasználó-hitelesítési felületét valósítja meg, amely lehetővé teszi a felhasználók számára a rendszerbe történő belépést. A komponens egy modal ablak formájában jelenik meg, amely a fő felület fölé kerül, így a felhasználó figyelme a bejelentkezési folyamatra összpontosul. A modal megjelenítése során a háttér görgetése letiltásra kerül, ezzel is javítva a felhasználói élményt.

A komponens elsődleges feladata az email cím és jelszó bekérése, valamint ezek kliens oldali ellenőrzése. A validáció biztosítja, hogy a felhasználó ne tudjon üres mezőkkel bejelentkezési kérést indítani. Az adatok kezelése useRef hook segítségével történik, amely lehetővé teszi az input mezők értékeinek közvetlen elérését a beküldés pillanatában.

Sikeres validációt követően a komponens HTTP POST kérést küld a backend felé a megfelelő végpontra. A kérés JSON formátumban tartalmazza a felhasználó által megadott adatokat. A backend válasza alapján a frontend eldönti a további működést: sikeres bejelentkezés esetén a kapott token eltárolásra kerül a böngésző localStorage-ében, valamint egy lejáratí idő is

beállításra kerül. Ezt követően egy esemény (authChanged) kerül kiváltásra, amely értesíti az alkalmazás többi részét a felhasználó állapotának megváltozásáról.

Sikertelen bejelentkezés esetén a rendszer azonnali visszajelzést ad a felhasználónak hibaüzenet formájában, így biztosítva az egyértelmű és gyors hibakezelést.

A komponens állapotkezelése több elemből áll, ide tartozik a mezők validációs állapota, a felhasználói interakciók nyomon követése, valamint a hibák kezelése. Ezek együttesen biztosítják a megbízható és felhasználóbarát működést.

3.5.2 Registration komponens

A Registration komponens az új felhasználók rendszerbe történő regisztrációját biztosítja. Feladata, hogy egy átlátható és biztonságos módon gyűjtse be a szükséges adatokat, majd továbbítsa azokat a backend felé feldolgozásra. A komponens a Login felülethez hasonlóan modal ablak formájában jelenik meg, így egységes felhasználói élményt nyújt, és könnyen elérhető a bejelentkezési felületről.

A komponens elsődleges szerepe a felhasználói adatok – például email cím és jelszó – bekérése, valamint ezek kliens oldali ellenőrzése. A validáció során a rendszer ellenőrzi az adatok helyességét, különösen a jelszó és a jelszó megerősítésének egyezését. Hibás vagy hiányos adatok esetén a felhasználó azonnali visszajelzést kap, így elkerülhető a hibás adatok elküldése.

A regisztrációs folyamat során a felhasználó által megadott adatok egy HTTP kérés formájában kerülnek elküldésre a backend felé. A backend feldolgozza az adatokat, ellenőrzi azok egyediségét (például hogy az email cím már létezik-e), majd siker esetén létrehozza az új felhasználót.

A komponens szorosan együttműködik a Login felülettel, lehetőséget biztosítva a két nézet közötti gyors váltásra. Ezáltal a felhasználó könnyedén navigálhat a bejelentkezés és a regisztráció között anélkül, hogy elhagyná az aktuális oldalt.

3.5.3 Filter komponens

A Filter komponens az alkalmazásban megjelenített adatok szűréséért felelős, lehetővé téve a felhasználók számára, hogy különböző szempontok alapján szűkítsék a találatokat. Célja, hogy gyors és hatékony keresési élményt biztosítson, valamint megkönnyítse a releváns elemek megtalálását nagyobb adathalmazok esetén.

A komponens többféle szűrési lehetőséget kínál, például kategória, márka vagy ár alapján. A felhasználó ezek közül tetszőlegesen választhat, így akár több feltételt is kombinálhat egyidejűleg. Az input mezők és kiválasztott opciók állapotát a komponens belső state-ben tárolja, ami biztosítja a dinamikus és azonnali frissítést.

A kategóriák kiválasztása például checkbox alapú megoldással történik, ahol a felhasználó több elemet is kijelölhet. A komponens kezeli, hogy egy adott elem már ki van-e választva, és ennek megfelelően hozzáadja vagy eltávolítja azt a kiválasztott kategóriák listájából. Az ár szerinti szűrés minimum és maximum értékek megadásával történik, míg más mezők egyszerű inputként működnek.

A szűrési feltételek minden változás esetén frissítésre kerülnek, így a rendszer valós időben reagál a felhasználói interakciókra. A komponens nem közvetlenül végzi az adatok szűrését, hanem egy callback függvényen keresztül továbbítja a kiválasztott paramétereket a szülő komponensnek.

4. Tesztelői dokumentáció

4.1 Tesztelés célja, menete

A következő tesztek a projektünk legfőbb és legfontosabb funkcióit foglya tesztelni. Ennek célja az esetleges hibák feltárása, azok kijavítása és ledokumentálása. A tesztek két csoportra vannak bontva: manuális tesztesetekre és automatizált tesztesetekre. A manuális tesztelés során manuálisan, lépésről lépésre, tégláról téglára megyünk végig az egyes funkciók folyamatán és lépésein. A manuális tesztesetek a bejelentkezés, regisztráció, hirdetés létrehozása és hirdetés módosítása funkciókat teszteli, míg az automata tesztek közül egy id alapú lekérést és egy hirdetés kosárba történő helyezését teszteli le.

4.2 Tesztelési környezet

A teszteléshez a projektet dockerizálva futtatjuk. Ehhez szükségünk van a **Docker** elindítására. Ezután a projekt törzsmappájában ki kell adnunk a **docker compose up –build** parancsot, ami elindítja a dockerizáció folyamatát. Amint ez befejeződik meg kell nyitnunk egy modern böngészőt. Mi a jelenlegi tesztek **Chrome** segítségével fogjuk megvalósítani. A projekt a kereső mezőbe **localhost** kulcsszót beírva tudjuk elérni és használni. A mindezekhez szükséges folyamatok **Windows 11** operációs rendszer alatt futnak.

4.3 Manuális tesztek

Manuális tesztesetek				
Azonosító	Funkció	Elvárt eredmény	Tényleges eredmény	Státusz
1	Bejelentkezés	Sikeres bejelentkezés	Sikeres bejelentkezés	Sikeres
2	Bejelentkezés	Sikertelen bejelentkezés	Sikertelen bejelentkezés	Sikeres
3	Bejelentkezés	Sikertelen bejelentkezés	Sikertelen bejelentkezés	Sikeres

4	Bejelentkezés	Sikertelen bejelentkezés	Sikertelen bejelentkezés	Sikeres
5	Regisztráció	Sikeres regisztráció	Sikeres regisztráció	Sikeres
6	Regisztráció	Sikertelen regisztráció	Sikertelen regisztráció	Sikeres
7	Új hirdetés létrehozása	Sikeres hirdetés létrehozás	Sikeres hirdetés létrehozás	Sikeres
8	Új hirdetés létrehozása	Sikertelen hirdetés létrehozás	Sikertelen hirdetés létrehozás	Sikeres

4.2.1 Bejelentkezés tesztek

Tesztazonosító: 1

Leírás: Sikeres bejelentkezés a megfelelő adatok megadása után.

Lépések:

1. A főoldalról a bejelentkezés oldalra navigálok.
2. Megadok egy létező email címet.
3. Megadom az email címhez tartozó jelszót.
4. Rákattintok a Belépés gombra.
5. A rendszer a főoldalra dob, a bejelentkezés sikeresen megtörtént.

Elvárt eredmény: Sikeres bejelentkezés.

Tényleges eredmény: Sikeres bejelentkezés.

Teszt státusza: Sikeres.

Tesztazonosító: 2

Leírás: Sikertelen bejelentkezés nem létező email cím megadásával.

Lépések:

1. A főoldalról a bejelentkezés felületre navigálok.
2. Megadok egy nem létező email címet.
3. Megadok egy létező jelszót.
4. Rákattintok a belépés gombra.
5. Alert message értesít a sikertelen bejelentkezésről. Üzenet: Hibás e-mail cím vagy jelszó!

Elvárt eredmény: Sikertelen bejelentkezés.

Tényleges eredmény: Sikertelen bejelentkezés, alert message, hibaüzenet.

Teszt státusza: Sikeres.

Tesztazonosító: 3

Leírás: Sikertelen bejelentkezés hibás jelszó megadásával.

Lépések:

1. A főoldalról a bejelentkezés felületre navigálok.
2. Megadok egy létező email címet.
3. Megadok egy hibás jelszót.
4. Rákattintok a belépés gombra.
5. Alert message értesít a sikertelen bejelentkezésről. Üzenet: Hibás e-mail cím vagy jelszó!

Elvárt eredmény: Sikertelen bejelentkezés.

Tényleges eredmény: Sikertelen bejelentkezés, alert message, hibaüzenet.

Teszt státusza: Sikeres.

Tesztazonosító: 4

Leírás: Sikertelen bejelentkezés a mezők kitöltetlenül hagyásával.

Lépések:

1. A főoldalról a bejelentkezés felületre navigálok.
2. Nem adok email címet.
3. Nem adok meg jelszót.
4. Rákattintok a belépés gombra.
5. Alert message értesít a sikertelen bejelentkezésről. Üzenet: Töltsd ki az összes mezőt!

Elvárt eredmény: Sikertelen bejelentkezés.

Tényleges eredmény: Sikertelen bejelentkezés, alert message, hibaüzenet.

Teszt státusza: Sikeres.

4.2.2 Regisztráció tesztek

Tesztazonosító: 5

Leírás: Sikeres regisztráció a megfelelő adatok megadásával.

Lépések:

1. A regisztrációs felületre navigálok.
2. Helyesen megadom a nevet.
3. Helyesen kitöltöm a felhasználónév mezőt.
4. Helyesen megadom az email címet a megfelelő formátumban.
5. Megadok egy helyes jelszót.
6. Megismétlem a jelszó megadását.
7. Megadom a telefonszámot a megfelelő formátumban.
8. Kiválasztom a születési időt a naptárból.
9. A Regisztráció gombra kattintok.
10. A rendszer a főoldalra dob, sikeresen regisztrál és automatikusan bejelentkeztet.

Elvárt eredmény: Sikeres regisztráció.

Tényleges eredmény: Sikeres regisztráció.

Teszt státusza: Sikeres.

Tesztazonosító: 6

Leírás: Sikertelen regisztráció üresen hagyott mezőkkel.

Lépések:

1. A regisztrációs felületre navigálok.
2. Megadom helyesen a nevet.
3. Kihagyom a felhasználónév mezőt.
4. Az email címet ismételten helyesen adom meg.
5. Megadok egy jelszót.
6. Megismétlem a jelszó megadását helyesen.
7. Nem adok meg telefonszámot.
8. Kiválasztom a születési időt.

9. A Regisztráció gombra kattintok.
10. Alert message értesítést kapok a sikertelen regisztrációról. Üzenet:
Kérlek töltsd ki a mezőket helyesen.

Elvárt eredmény: Sikertelen regisztráció.

Tényleges eredmény: Sikertelen regisztráció, alert message, hibaüzenet.

Teszt státusza: Sikeres.

4.2.3 Új hirdetés tesztek

Tesztazonosító: 7

Leírás: Sikeres hirdetés létrehozás a megfelelő adatokkal.

Lépések:

1. Sikeresen bejelentkezek egy létező felhasználóval.
2. Az Új hirdetés felületre navigálok.
3. Helyesen megadom a termék nevét.
4. Kiválasztom a kategóriát a legördülő menüből.
5. Kiválasztom a márkát a legördülő menüből.
6. Kiválasztom a termék állapotát a legördülő menüből.
7. Helyesen megadom az árat.
8. Sikeresen kiválasztom a hirdetés képét.
9. Megadom a hirdetés részletes leírását.
10. A Hirdetés létrehozása gombra kattintok.
11. Alert message jelenik meg a sikeres hirdetés létrehozásról. Üzenet:
Hirdetés létrehozása sikeres.
12. A rendszer a Hirdetéseim felületre navigál.

Elvárt eredmény: Sikeres hirdetés létrehozás.

Tényleges eredmény: Sikeres hirdetés létrehozás, alert message.

Teszt státusza: Sikeres.

Tesztazonosító: 8

Leírás: Sikertelen hirdetés létrehozás mezők üresen hagyásával.

Lépések:

1. Sikeresen bejelentkezek egy létező felhasználóval.
2. Az Új hirdetés felületre navigálok.
3. Kihagyom a termék nevének megadását.
4. Sikeresen kiválasztom a kategóriát a legördülő menüből.
5. Sikeresen kiválasztom a márkát a legördülő menüből.
6. Nem adok meg árat.
7. Nem töltök fel képet.
8. Megadom a leírást.
9. Az Új hirdetés gombra kattintok.
10. Az oldal alján piros hibaüzenet jelenik meg. Üzenet: Kérjük, tölts ki minden kötelező mezőt!

Elvárt eredmény: Sikertelen hirdetés létrehozás.

Tényleges eredmény: Sikertelen hirdetés létrehozás, hibaüzenet.

Teszt státusza: Sikeres.

4.4 Automatizált tesztek

4.4.1 Az automatizált tesztekhez használt eszközök és technológiák

Az automatizált teszteknek két csoportja van a projektünkben: **E2E** (end to end) tesztek és **unit** tesztek.

Az E2E tesztek megvalósításához a **Cypress** nevű tesztelési eszközt alkalmazzuk backenden. A tesztek mappaszerkezete: Backstage-Gear (törzsmappa) -> backend mapppa -> cypress mappa -> e2e mappa. Ezen mappán belül foglal helyet 6 különböző tesztfájl, amik egyesével egy-egy nagyobb egységet tesztelnek pl.: advertisements.cy.ts, ami a hirdetésekhez kapcsolódó funkciók tesztelését végzi. A tesztek futtatásához a backend mappán belül ki kell adnunk az **npm cypress open** parancsot, ami automatikusan megnyitja a cypress grafikus felületét. Ott az **E2E Testing** lehetőséget

választva kiválaszthatjuk, melyik böngészővel szeretnénk futtatni a teszteket (a mi esetünkben ez most a **Chrome**). A **Start E2E testing in Chrome** lehetőségre kattintva megkezdhetjük az egyes egységek tesztelését. Fontos, hogy ezeket a teszteket nem az éles adatbázison, hanem egy tesztadatbázison (**seed.sql**) végezzük, ami a cypress mappán belül egy külön db mappában található.

A unit teszteket **Jest** segítségével valósítottuk meg. Ezen tesztek sikeres teljesítéséhez mindent **service rétegbe** szerveztünk, hogy így könnyítsük meg és tegyük átláthatóbbá a tesztelés folyamatát. Ezek a tesztek a backend mappában, azon belül is az src mappában vannak, mindegyik a hozzá tartozó controller és service réteg mellett, pl.: a category mappában foglal helyet a categoryService.ts, a categoryController.ts és a categoryService.test.ts, ez utóbbiban foglalnak helyet a tesztek. Ezen tesztek futtatásához a backend mappán belül ki kell adni az **npm test** parancsot.

A frontend automatizált tesztelése **Vitest**tel történik, közvetlenül a Vite konfigurációra építve. A vite.config.js fájl alapján a tesztek **jsdom** környezetben indulnak, globális Vitest API-val, a közös előkészítést pedig a vitest/setupTests.js fájl adja. Ez a megoldás lehetővé teszi, hogy a React felületet böngészőhöz hasonló DOM környezetben ellenőrizzük.

A DOM-vizsgálatokhoz a @testing-library/react, a felhasználói interakciók modellezéséhez a @testing-library/user-event, az olvasható állításokhoz pedig a @testing-library/jest-dom/vitest bővítmény tartozik. A vitest/testUtils.jsx és a setup állományok közös mockokat biztosítanak például az auth segédekhez és a react-router-dom navigációhoz. Mindez a frontend könyvtárból a **npm test** paranccsal indítható.

4.4.2 Példák automatizált tesztekre

4.4.2.1 E2E példateszt

Leírás: Lekéri egy adott felhasználó beérkező üzenetét az id alapján.

1. Rajz: E2E teszteset példa üzenet lekérésnél

```
describe('GET  
/backstagegear/me/incoming_messages/:messageId', () => {
```

```

it('should return the datas of the exact message of a user',
() => {
  cy.request({
    method: 'GET',
    url: '/backstagegear/me/incoming_messages/2',
    headers: { 'x-access-token': messageToken }
  }).then(res => {
    expect(res.status).to.eq(200);
    expect(res.body[0]).to.have.property('id', 2);
  })
});
});

```

A teszt lekéri egy felhasználó bejövő hirdetését az üzenet egyedi azonosítója alapján, továbbá ellenőrzi, hogy az így kapott státuszkód 200-e (OK), és ha igen, akkor megnézi, hogy ténylegesen a 2-es azonosítójú hirdetést kaptuk-e vissza. A tesztfájlban további helyet kapnak olyan tesztek is, ahol a messageId helytelen megadásával a lekérés lehetséges hibáit ellenőrizzük. Továbbá, amint az látható, üzenetküldéshez a felhasználónak be kell jelentkeznie, így a fájlban helyet kapnak olyan tesztek is, ahol a messageToken meglétét ellenőrizzük.

4.4.2.2 Unit példateszt

Leírás: Lekéri, hogy egy adott felhasználó admin-e vagy sem.

2. Rajz: Unit teszteset példa admin szerepkör lekérdezésnél

```
describe("getIsAdminById", () => {
  let req: any;
  let res: any;
  beforeEach(() => {
    req = { user: { id: "1" } };
    res = {
      status: jest.fn().mockReturnThis(),
      send: jest.fn(),
      json: jest.fn()
    };
    jest.clearAllMocks();
  });
  it("should return 200 and is_admin true if user is admin",
  async () => {
    jest.spyOn(userService,
    "getIsAdminService").mockResolvedValue(true);
    await getIsAdminById(req, res);
    expect(res.status).toHaveBeenCalledWith(200);
    expect(res.send).toHaveBeenCalledWith({ is_admin: true });
  });
  it("should return 200 and is_admin false if user is not
  admin", async () => {
    jest.spyOn(userService,
    "getIsAdminService").mockResolvedValue(false);
    await getIsAdminById(req, res);
    expect(res.status).toHaveBeenCalledWith(200);
    expect(res.send).toHaveBeenCalledWith({ is_admin: false
  });
  });
  it("should return 404 if user not found", async () => {
    jest.spyOn(userService,
    "getIsAdminService").mockResolvedValue(null);
    await getIsAdminById(req, res);
    expect(res.status).toHaveBeenCalledWith(404);

    expect(res.send).toHaveBeenCalledWith(expect.stringContaining
    ("Nem létezik ilyen azonosítójú felhasználó"));
  });
  it("should return 500 on error", async () => {
    jest.spyOn(userService,
    "getIsAdminService").mockRejectedValue(new Error("fail"));
    await getIsAdminById(req, res);
    expect(res.status).toHaveBeenCalledWith(500);

    expect(res.send).toHaveBeenCalledWith(expect.stringContaining
    ("Szerver hiba"));
  });
});
```

A teszt itt egy, olyan függvényt tesztl, ami true/false értékkel tér vissza attól függően, hogy az adott felhasználó admin jogosultsággal rendelkezik-e vagy sem. A teszt ellenőrzi, hogy amikor helyesen tér vissza valamilyen értékkel pl.: true vagy false, akkor a státusz kód 200-e (OK). Teszteli továbbá, hogyha a felhasználó nem létezik, ilyenkor a backend 404-es (Not Found)

státuszkóddal tér vissza. Továbbá ellenőrzi, hogyha valamilyen váratlan hiba lép fel szerveroldalon, ilyenkor 500-as (Internal Server Error) státuszkóddal tér vissza a backend.

4.4.2.3 Frontend komponensszintű példa

A `login.test.jsx` és a `registration.test.jsx` tesztek azt ellenőrzik, hogy a felhasználó az űrlapmezők kitöltésével el tudja indítani a bejelentkezési és regisztrációs folyamatot, majd a komponens a megfelelő REST végpontot hívja meg. A `filter.test.jsx` külön azt vizsgálja, hogy a Filter komponens betölti-e a kategória- és márkaadatokat, majd módosítás esetén továbbadja-e a szülő komponens felé az új szűrési állapotot.

A kisebb űrlapelemekre és segédkomponensekre is külön tesztek készültek. Az `inputField.test.jsx`, `AdFormFields.test.jsx`, `AdFormActions.test.jsx` és `selectOption.test.jsx` fájlok biztosítják, hogy a mezők, validációk és űrlapgombok önmagukban is megbízhatóan működjenek.

4.4.2.4 Frontend oldalszintű példa

A `NewAd.page.test.jsx` négy különböző scenáriót futtat: ellenőrzi a kategória- és márkalista betöltését, a kötelező mezők hiányának kezelését, a sikeres hirdetésfeladást és a szerverhiba megjelenítését. A `Message.page.test.jsx` egy teljesebb folyamatot modellez, mert betölti a beszélgetéseket, majd ugyanazon tesztben ellenőrzi a válaszküldést és a beszélgetéstörlést is.

A `Profile.page.test.jsx` a profil oldal három fontos ágát fedi le: az adatbetöltést, a profilfrissítés sikeres és hibás válaszait, valamint a jelszóváltó modal működését. Ezt egészítik ki a `Home.page.behavior.test.jsx`, `Products.page.behavior.test.jsx`, `ViewAd.page.test.jsx` és `ReportedAds.page.behavior.test.jsx` állományok, amelyek a keresési, részletező és adminisztrátori oldalak üzleti viselkedését ellenőrzik.

5. Felhasználói dokumentáció

Ebben a fejezetben bemutatjuk a projektben található funkciókat lépésről lépésre, és leírjuk az ezekhez szükséges részletes navigációt az oldalon.

5.1 Böngészés

A felhasználót a főoldal fogadja, amin a következők találhatók fentről lefele: navigációs menü, kereső mező, kategóriák, legfrissebb hirdetések, lábjegyzet. Az oldal jobb felső sarkában található egy profil ikon, amelyre kattintva a még be nem jelentkezett felhasználó olyan lehetőségeket érhet el, mint a bejelentkezés vagy regisztráció.

Az oldalon történő böngészéshez nem kell a felhasználónak bejelentkeznie vagy regisztrálnia. Az oldalon a keresés többféleképpen történhet.

Amennyiben a felhasználó kulcsszavak szerint szeretne egy adott hirdetésre keresni, azt a navigációs menü alatt található kereső mező segítségével tudja megtenni. A felhasználó beírja a keresendő kulcsszavat a mezőbe és Enter billentyű lenyomásával megkapja a szűrt adatokat.

Egy másik lehetőség a kategóriák szerinti szűrés, amit a felhasználó a kereső menü alatt található kategóriák gombok valamelyikének megnyomásával érhet el. Ilyenkor a hirdetések kategóriák szerint automatikusan szűrve lesznek. Mind ezek után a felhasználó tovább szűkítheti a termékek számát az oldal bal oldalán található szűrők segítségével. Itt lehetőség van tovább szűrni árra, állapotra és márkára.

Amennyiben a felhasználót komolyabban is érdekel egy hirdetés, akkor a hirdetésre kattintva megtekintheti annak részletesebb adatait.

5.2 Regisztráció

A főoldal jobb felső sarkában található egy profil ikon. Erre kattintva megjelenik majd a Belépés felület. A felület alján található a Regisztráció gomb, erre kattintva megnyílik a regisztrációs felület. A profil adatainak megadásánál minden mezőt kötelező kitölteni, ennek elmulasztását az oldal egy hibaüzenettel is jelzi, valamint azt is, ha a megadott adat formátuma vagy hossza nem megfelelő. Miután a

felhasználó kitöltötte a mezőket a form alatt található Regisztráció gombra kattintva hozhat létre egy új profilt. Sikeres regisztráció után az oldal automatikusan bejelentkezteti és a főoldalra navigálja a felhasználót.

5.3 Bejelentkezés

A főoldal jobb felső sarkában található egy profil ikon. Erre kattintva megjelenik majd a Belépés felület. A bejelentkezéshez meg kell adni egy email címet és a profilhoz tartozó jelszót. Amennyiben a felhasználó nem adja meg valamelyik adatot arra az oldal egy hibaüzenettel figyelmeztet. Szintén, hogyha az adatokat nem megfelelően adjuk meg (elgépeljük a jelszót vagy az email címet), az oldal erre is figyelmeztet. Miután helyesen kitöltöttük a mezőket a felület alján található Belépés gombra kattintva a felhasználó sikeresen be tud jelentkezni. Az oldal automatikusan a főoldalra navigál.

5.4 Hirdetés kívánság listára helyezése

Egy hirdetés kívánság listára helyezéséhez a felhasználónak először be kell jelentkeznie egy már létező profillal. A listára tűzés többféleképpen is megvalósítható.

A hirdetés összesített megjelenítésénél található Kívánságlistára tűzés gombra kattintva lehetőség van a hirdetés listára helyezésére. Ezt megtehetjük például a főoldalon a legfrissebb hirdetések esetén, illetve a szűrt hirdetések felületén is.

A másik lehetőség a hirdetés adatainak részletes megtekintésénél lehetséges, szintén a Kívánságlistára tűzés gombra kattintva.

Fontos, hogy egy felhasználó nem tűzhet listára egy adott hirdetést egynél többször és nem tűzheti listára a saját hirdetéseit.

5.5 Kívánságlista megtekintése

Az oldal tetején lévő navigációs sáv jobb oldalán található lista ikonra kattintva a felhasználó megtekintheti a már listázott hirdetéseket. Egy zöld keretben található szám jelzi is az oldalon a listában található hirdetések számát.

Amennyiben Egy felhasználó már nem kíván egy hirdetést a listán tartani, egyszerűen az Eltávolítás a kívánságlistáról gombra kattintva törölheti a listáról a hirdetést.

5.6 Új hirdetés létrehozása

Amennyiben a felhasználó egy új hirdetést szeretne létrehozni, azt a navigációs sáv jobb oldalán található ikon gombra kattintva, az Új hirdetés lehetőséget választva teheti meg.

A gomb elnavigál a hirdetés létrehozásához lehetőséget biztosító oldalra. Ezen az oldalon a kép feltöltésén kívül minden mezőt ki kell tölteni: Termék neve, Kategória, Márka, Állapot, Ár, Leírás. Néhány mezőnek alapértelmezetten vannak értékei: Kategória, Márka, Állapot.

A termék neve mezőbe a felhasználónak magának kell begépelnie az adatot.

A Kategória, Márka és az Állapot leugró menük, ahol a felhasználó az adatbázisban található lehetőségekből választhat.

Az Ár mezőbe a felhasználónak magának kell begépelnie a termék árát.

A Kép feltöltése lehetőség nem kötelező, amennyiben a felhasználó nem tölt fel képet, a rendszer automatikusan egy alapértelmezett képet rendel a hirdetéshez.

A Leírás mezőbe a felhasználónak magának kell begépelnie a hirdetés leírását.

Amennyiben a felhasználó hiányosan vagy hibásan adja meg a mezők tartalmát, az oldal egy hibaüzenettel jelzi azt.

Miután a felhasználó sikeresen kitöltött minden kötelező mezőt, az oldal alján található Hirdetés létrehozása gombra kattintva létrehozhatja a hirdetést. Ezután az oldal automatikusan a Hirdetésem oldalra navigál.

5.7 Hirdetések módosítása, törlése, jelentése

Abban az esetben, ha a felhasználó frissíteni szeretné egy hirdetése adatait, azt a profil ikonra, majd a Hirdetésem lehetőségre kattintva teheti meg.

Az adatok módosítása esetében a Módosítás gombra kattintva az oldal a Hirdetés szerkesztése oldalra navigál. Itt a felhasználó láthatja az eredetileg megadott adatokat. Amennyiben a felhasználó üresen hagy egy mezőt, azt az oldal egy hibaüzenettel jelzi. Miután sikeresen módosította a felhasználó a hirdetés adatait, az oldal alján található Módosítások mentése gombra kattintva véglegesíthetjük a módosításokat. Ezután az oldal a Hirdetéseim oldalra navigál vissza.

Ha a felhasználó törölni szeretné a hirdetését, azt szintén a Hirdetéseim oldalon teheti meg. A hirdetésre kattintva megjelenik a hirdetés részletes nézete. Ott található egy piros Hirdetés törlése gomb. Erre kattintva véglegesen törölhetjük a hirdetést. Ezután az oldal a Hirdetéseim oldalra navigál vissza.

A felhasználónak lehetősége van egy hirdetés jelentésére, ha úgy érzi annak tartalma irreleváns, sértő vagy megbotránkoztató. Ezt az adott hirdetésre kattintva – annak részletes nézetében - tehetjük meg. Ott található majd többek között egy piros Hirdetés jelentése gomb, erre kattintva sikeresen jelenthetjük a hirdetést. Fontos, hogy a saját hirdetésünket nem jelenthetjük.

5.8 Üzenetek kezelése

Ha egy felhasználó üzenetben szeretné érdeklődését kifejezni egy adott hirdetés iránt, azt egy adott hirdetés részletes nézetében tehetjük meg a kék Érdeklődés gomb megnyomásával. Ekkor az oldal az Üzeneteim oldalra navigál, ahol lehetőségünk van üzenetet küldeni az adott hirdetés hirdetőjének.

Ha az üzenet mezőbe begépettük az elküldendő üzenetet, akkor a zöld Küldés gombra kattintva egy új beszélgetést indíthatunk az adott felhasználóval. Amennyiben mégsem szeretnénk üzenetet küldeni azt a kék Mégsem gombra kattintva vonhatjuk vissza.

Az üzenet sikeres elküldése után az oldal az Üzeneteim oldalra navigál, ahol a korábbi beszélgetéseinket is megtalálhatjuk. Ha valakinek választ szeretnénk küldeni akkor a felhasználóhoz tartozó mezőre vagy profilképre, majd a zöld Válasz gombra kattintva tehetjük meg.

Amennyiben törölni szeretnénk egy beszélgetést, azt szintén a felhasználóhoz tartozó mezőre vagy profilképre, majd a piros Beszélgetés gombra kattintva tehetjük meg.

Az Üzeneteim felületet a profil ikonra, majd az Üzenetek lehetőségre kattintva szintén elérhető.

5.9 Profil kezelése

A felhasználó saját profiljának módosításához a profil ikonra, majd a Profil lehetőségre kattintva érheti el a Profil szerkesztése oldalt. Itt láthatók a profilhoz megadott adatok.

Abban az esetben, ha a felhasználó módosítani szeretné az általános adatait: Felhasználónév, Telefonszám, Profilkép, azt az Adatok módosítása lehetőségre kattintva teheti meg. Ilyenkor szabadon átírhatja és frissítheti az adatait. Amikor végzett, a Mentés gombra kattintva véglegesítheti a módosításokat, ha mégsem szeretné menteni a változtatásokat, azt a Mégse gombra kattintva teheti meg.

Akkor, ha a felhasználó a jelszavát szeretné módosítani, azt a Jelszó módosítása gombra kattintva teheti meg. Ilyenkor felugrik a Jelszó módosítása felület, ahol három mező található. A Jelenlegi jelszó mező, ahol a felhasználónak éppen aktuális jelszavát kell megadnia, az Új jelszó mező, ahol a felhasználónak az új jelszavát kell begépelni és az Új jelszó újra mező, ahol a felhasználónak megint meg kell adnia az új jelszavát, ezzel megerősítve döntését. Amennyiben mégsem szeretné módosítani a jelszót, a felhasználó a folyamatot egy félrekattintással vagy a felület jobb felső sarkában található X-re kattintva szakíthatja meg.

A felhasználónak lehetősége van törölni is a profilját, amit a Profil szerkesztése oldalon a piros Profil törlése gomb megnyomásával teheti meg.

5.10 Profil értékelése

A felhasználónak lehetősége van más profilok értékelésére. Ezt egy hirdetés részletes nézetében teheti meg a profil kép alatt található kék - vagy piros + gombok kattintásával. Egy profilt, vagy csak egy – (down) vagy csak egy + (up) megnyomásával értékelhetünk. Saját profilunkat nincs lehetőség értékelni.

5.11 Kijelentkezés

Kijelentkezésre is van lehetőség, ezt a profil ikonra majd a piros Kilépés gombra kattintva tehetjük meg.

6. Összefoglalás

A projektünk célja, hogy lehetőséget biztosítson mindenki számára, hogy a már nem használt audio eszközét, legyen az egy hangszer, erősítő, stúdiófelszerelés vagy hangosítási technika, kényelmesen és gyorsan hirdetni tudja a felületünkön. Magánszemélyek, gyűjtők, kezdő vagy már professzionális zenészek, technikusok, itt mindenki megtalálhatja azt, amit keres. Továbbá ezzel is szeretnénk támogatni a zene világát és az abban tevékenykedőket. Mindemellett fontos célkitűzésünk az újrahasznosítás támogatása is, hiszen egyetlen hangszer sem arra született, hogy csendben porosodjon egy sarokban vagy egy szekrény mélyén várja, hogy újra ragyogjon. Minden használt termék megérdemel egy második esélyt.

6.1 Rövid értékelés:

A projekt során sikerült egy egységes és jól megosztott munkafolyamatot követnünk. A szerepek felosztása sikeresnek bizonyult, a backend és a frontend megírását felosztottuk így lehetővé téve az egységes megvalósításukat. Ha volt kérdés, kérés vagy ötlet a másik fél irányába azt mindig sikerrel tudtuk kommunikálni egymás felé, és ha szükség volt rá, mindig kíségtettük egymást egy-egy kisebb-nagyobb munkafolyamat során.

A célunk egy jól és stabilan működő projekt megvalósítása volt egy kiegyensúlyozott csapatmunka során. Ezen kitűzött célunkat sikeresen teljesítettük.

7. Kitekintés

A jövőbeni továbbfejlesztés során lehetne lehetőség több kategória és márka felvételére az adatbázisba, illetve több hasznos funkció implementálása

pl.: kétlépcsős azonosítás bevezetése, valamint egy mobilalkalmazás lefejlesztése.

8. Irodalomjegyzék/Bibliográfia

HTTP státuszkódok megnevezése: <https://http.cat>

Frontenden használt ikonok: <https://icons8.com>

Frontenden használt színminta kialakítása: <https://www.realtimecolors.com>

Egyéb segítségék:

<https://www.w3schools.com>

<https://stackoverflow.com/questions>

9. Mellékletek

API végpontok részletes dokumentációja: <https://github.com/Zatony/Backstage-Gear/blob/main/mellekletek/API%20dokument%C3%A1ci%C3%B3.docx>

Manuális tesztek részletes dokumatnációja: https://github.com/Zatony/Backstage-Gear/blob/main/mellekletek/teszt_dokument%C3%A1ci%C3%B3.xlsx

Diagramok:

- ER diagram: <https://github.com/Zatony/Backstage-Gear/blob/main/mellekletek/ER.drawio.pdf>
- ERD diagram: <https://github.com/Zatony/Backstage-Gear/blob/main/mellekletek/ERD.drawio.pdf>
- Sequence diagram: https://github.com/Zatony/Backstage-Gear/blob/main/mellekletek/BG_SequenceDiagram.drawio.pdf
- Use Case diagram: <https://github.com/Zatony/Backstage-Gear/blob/main/mellekletek/UseCase.drawio.pdf>